

✓ Customer Segmentation Analysis for E-commerce

This notebook aims to perform customer segmentation analysis for an e-commerce company by analyzing customer behavior and purchase patterns. The goal is to group customers into distinct segments to inform targeted marketing strategies and improve customer satisfaction.

```
import pandas as pd
```

```
# Load the dataset from the /content/sample_data/ directory  
# The file 'ifood_df.csv' is found in the Colab environment's sample_data.  
data = pd.read_csv('/content/sample_data/ifood_df.csv')
```

```
print("Dataset 'ifood_df.csv' loaded successfully from /content/sample_data/  
# Display the first 5 rows of the dataset  
display(data.head())
```

Dataset 'ifood_df.csv' loaded successfully from /content/sample_data/!

	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts
0	58138.0	0	0	58	635	88	546
1	46344.0	1	1	38	11	1	6
2	71613.0	0	0	26	426	49	127
3	26646.0	1	0	26	11	4	20
4	58293.0	1	0	94	173	43	118

5 rows × 39 columns

```
# Display general information about the dataset  
display(data.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2205 entries, 0 to 2204
Data columns (total 39 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Income                                2205 non-null   float64
1   Kidhome                               2205 non-null   int64
2   Teenhome                              2205 non-null   int64
3   Recency                               2205 non-null   int64
4   MntWines                              2205 non-null   int64
5   MntFruits                             2205 non-null   int64
6   MntMeatProducts                       2205 non-null   int64
7   MntFishProducts                       2205 non-null   int64
8   MntSweetProducts                      2205 non-null   int64
9   MntGoldProds                          2205 non-null   int64
10  NumDealsPurchases                     2205 non-null   int64
11  NumWebPurchases                       2205 non-null   int64
12  NumCatalogPurchases                   2205 non-null   int64
13  NumStorePurchases                     2205 non-null   int64
14  NumWebVisitsMonth                     2205 non-null   int64
15  AcceptedCmp3                          2205 non-null   int64
16  AcceptedCmp4                          2205 non-null   int64
17  AcceptedCmp5                          2205 non-null   int64
18  AcceptedCmp1                          2205 non-null   int64
19  AcceptedCmp2                          2205 non-null   int64
20  Complain                              2205 non-null   int64
21  Z_CostContact                         2205 non-null   int64
22  Z_Revenue                             2205 non-null   int64
23  Response                              2205 non-null   int64
24  Age                                    2205 non-null   int64
25  Customer_Days                         2205 non-null   int64
26  marital_Divorced                      2205 non-null   int64
27  marital_Married                       2205 non-null   int64
28  marital_Single                        2205 non-null   int64
29  marital_Together                      2205 non-null   int64
30  marital_Widow                         2205 non-null   int64
31  education_2n Cycle                    2205 non-null   int64
32  education_Basic                       2205 non-null   int64
33  education_Graduation                  2205 non-null   int64
34  education_Master                      2205 non-null   int64
35  education_PhD                         2205 non-null   int64
36  MntTotal                              2205 non-null   int64
37  MntRegularProds                       2205 non-null   int64
38  AcceptedCmpOverall                    2205 non-null   int64
dtypes: float64(1), int64(38)
memory usage: 672.0 KB
None

```

✓ Data Cleaning and Preprocessing

```

# Check for duplicate rows
duplicate_rows = data.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_rows}")

if duplicate_rows > 0:

```

```
data.drop_duplicates(inplace=True)
print("Duplicate rows removed.")
```

Number of duplicate rows: 184
Duplicate rows removed.

```
# Inspect the 'Income' column for non-positive values
non_positive_income = data[data['Income'] <= 0]
print(f"Number of rows with non-positive income: {len(non_positive_income)}")

# Display statistics for the 'Income' column
display(data['Income'].describe())
```

Number of rows with non-positive income: 0

	Income
count	2021.000000
mean	51687.258783
std	20713.046401
min	1730.000000
25%	35416.000000
50%	51412.000000
75%	68274.000000
max	113734.000000

dtype: float64

From the `data.info()` output, we can see that all columns have 2205 non-null entries, which matches the total number of rows. This means there are no missing values in this dataset. However, we should still check for columns with only one unique value, as these would not contribute to the analysis.

```
# Identify and drop columns with only one unique value (constant columns)
constant_columns = [col for col in data.columns if data[col].nunique() == 1]
print(f"Constant columns: {constant_columns}")

if constant_columns:
    data.drop(columns=constant_columns, inplace=True)
    print(f"Dropped constant columns: {constant_columns}")
else:
    print("No constant columns found.")

# Display general information about the dataset after dropping constant columns
display(data.info())
```

```

Constant columns: ['Z_CostContact', 'Z_Revenue']
Dropped constant columns: ['Z_CostContact', 'Z_Revenue']
<class 'pandas.core.frame.DataFrame'>
Index: 2021 entries, 0 to 2204
Data columns (total 37 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Income                                2021 non-null   float64
1   Kidhome                              2021 non-null   int64
2   Teenhome                             2021 non-null   int64
3   Recency                              2021 non-null   int64
4   MntWines                             2021 non-null   int64
5   MntFruits                            2021 non-null   int64
6   MntMeatProducts                      2021 non-null   int64
7   MntFishProducts                      2021 non-null   int64
8   MntSweetProducts                    2021 non-null   int64
9   MntGoldProds                        2021 non-null   int64
10  NumDealsPurchases                    2021 non-null   int64
11  NumWebPurchases                      2021 non-null   int64
12  NumCatalogPurchases                  2021 non-null   int64
13  NumStorePurchases                    2021 non-null   int64
14  NumWebVisitsMonth                    2021 non-null   int64
15  AcceptedCmp3                         2021 non-null   int64
16  AcceptedCmp4                         2021 non-null   int64
17  AcceptedCmp5                         2021 non-null   int64
18  AcceptedCmp1                         2021 non-null   int64
19  AcceptedCmp2                         2021 non-null   int64
20  Complain                             2021 non-null   int64
21  Response                             2021 non-null   int64
22  Age                                  2021 non-null   int64
23  Customer_Days                        2021 non-null   int64
24  marital_Divorced                     2021 non-null   int64
25  marital_Married                      2021 non-null   int64
26  marital_Single                       2021 non-null   int64
27  marital_Together                     2021 non-null   int64
28  marital_Widow                        2021 non-null   int64
29  education_2n Cycle                   2021 non-null   int64
30  education_Basic                      2021 non-null   int64
31  education_Graduation                 2021 non-null   int64
32  education_Master                     2021 non-null   int64
33  education_PhD                        2021 non-null   int64
34  MntTotal                             2021 non-null   int64
35  MntRegularProds                      2021 non-null   int64
36  AcceptedCmpOverall                   2021 non-null   int64
dtypes: float64(1), int64(36)
memory usage: 600.0 KB
None

```

✓ Descriptive Statistics

```

# Display descriptive statistics for all numerical columns
display(data.describe())

```

	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits
count	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000	2021.000000
mean	51687.258783	0.443345	0.509649	48.880752	306.492331	26.300000
std	20713.046401	0.536196	0.546393	28.950917	337.603877	39.700000
min	1730.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	35416.000000	0.000000	0.000000	24.000000	24.000000	2.000000
50%	51412.000000	0.000000	0.000000	49.000000	178.000000	8.000000
75%	68274.000000	1.000000	1.000000	74.000000	507.000000	33.000000
max	113734.000000	2.000000	2.000000	99.000000	1493.000000	199.000000

8 rows × 37 columns

Feature Engineering

```
# Calculate 'Total_Spending': Sum of spending on all product categories
data['Total_Spending'] = data['MntWines'] + data['MntFruits'] + data['MntMeatProducts'] +
data['MntFishProducts'] + data['MntSweetProducts']

# Calculate 'Total_Purchases': Sum of purchases through all channels
data['Total_Purchases'] = data['NumDealsPurchases'] + data['NumWebPurchases'] +
data['NumCatalogPurchases'] + data['NumStorePurchases']

# Calculate 'Avg_Purchase_Value': Total_Spending / Total_Purchases
# Handle potential division by zero if Total_Purchases is 0
data['Avg_Purchase_Value'] = data.apply(lambda row: row['Total_Spending'] /
row['Total_Purchases'] if row['Total_Purchases'] != 0 else 0, axis=1)

# Display the head of the DataFrame with new features
display(data[['Total_Spending', 'Total_Purchases', 'Avg_Purchase_Value']].head(5))
```

	Total_Spending	Total_Purchases	Avg_Purchase_Value
0	1617	25	64.680000
1	27	6	4.500000
2	776	21	36.952381
3	53	8	6.625000
4	422	19	22.210526

```
# Display descriptive statistics for the newly created features
display(data[['Total_Spending', 'Total_Purchases', 'Avg_Purchase_Value']].describe())
```

	Total_Spending	Total_Purchases	Avg_Purchase_Value	
count	2021.000000	2021.000000	2021.000000	
mean	607.711034	14.897575	32.672236	
std	602.396167	7.598261	28.712492	
min	5.000000	0.000000	0.000000	
25%	69.000000	8.000000	9.600000	
50%	397.000000	15.000000	23.588235	
75%	1048.000000	21.000000	45.533333	
max	2525.000000	43.000000	186.500000	

Customer Segmentation: Data Preparation (Scaling and PCA)

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Also exclude 'AcceptedCmpOverall' and 'Response' for initial clustering
# as these are outcome variables.
features_for_clustering = data.select_dtypes(include=['int64', 'float64']).c

# Exclude specific columns that are not suitable for direct clustering or are
exclude_columns = [
    'Kidhome', 'Teenhome', 'Recency', 'Complain', 'Response', 'Age', 'Custom
    'marital_Divorced', 'marital_Married', 'marital_Single', 'marital_Togeth
    'education_2n Cycle', 'education_Basic', 'education_Graduation', 'educat
    'AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', 'AcceptedCmp1', 'Accepte
]

# Refine features_for_clustering by removing the excluded columns
features_for_clustering = [col for col in features_for_clustering if col not

X = data[features_for_clustering]

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Features scaled successfully.")
# Display the first 5 rows of the scaled data (as a DataFrame for readabilit
display(pd.DataFrame(X_scaled, columns=X.columns).head())

```

Features scaled successfully.

	Income	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetF
0	0.311511	0.973297	1.549924	1.728456	2.448975	
1	-0.258030	-0.875480	-0.637837	-0.728158	-0.648771	-1
2	0.962228	0.354075	0.569203	-0.177695	1.337431	-1
3	-1.209260	-0.875480	-0.562397	-0.664468	-0.502995	-1
4	0.318996	-0.395509	0.418323	-0.218638	0.152998	-1

```
# Apply PCA to reduce dimensionality
pca = PCA(n_components=2) # Reduce to 2 principal components for visualizati
X_pca = pca.fit_transform(X_scaled)

print(f"Explained variance ratio by principal components: {pca.explained_var

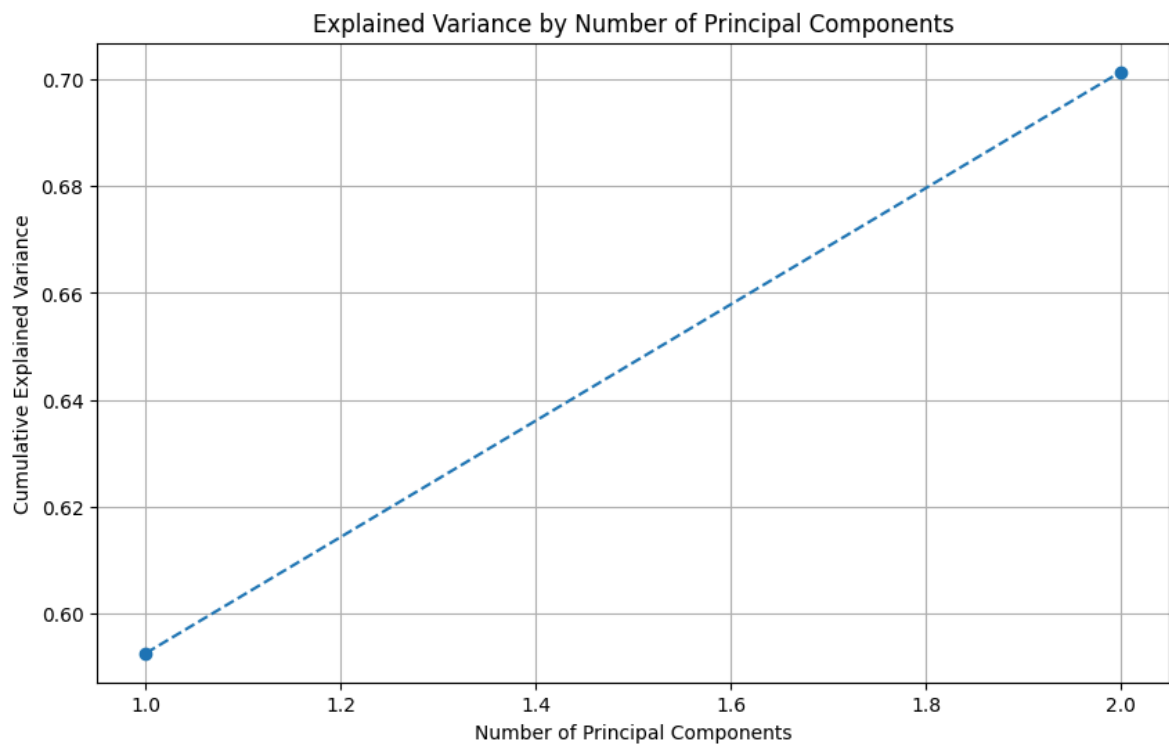
# Create a DataFrame for the PCA results
pca_df = pd.DataFrame(data=X_pca, columns=['Principal_Component_1', 'Princip

# Display the first 5 rows of the PCA DataFrame
display(pca_df.head())

# Plot the explained variance ratio
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(pca.explained_variance_ratio_) + 1), pca.explained_var
plt.title('Explained Variance by Number of Principal Components')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid(True)
plt.show()
```

Explained variance ratio by principal components: 0.70

	Principal_Component_1	Principal_Component_2	
0	4.993189	0.411334	
1	-3.079882	-0.943130	
2	1.824878	0.232295	
3	-3.095408	-0.448724	
4	-0.163483	1.004642	



✓ Customer Segmentation: K-Means Clustering

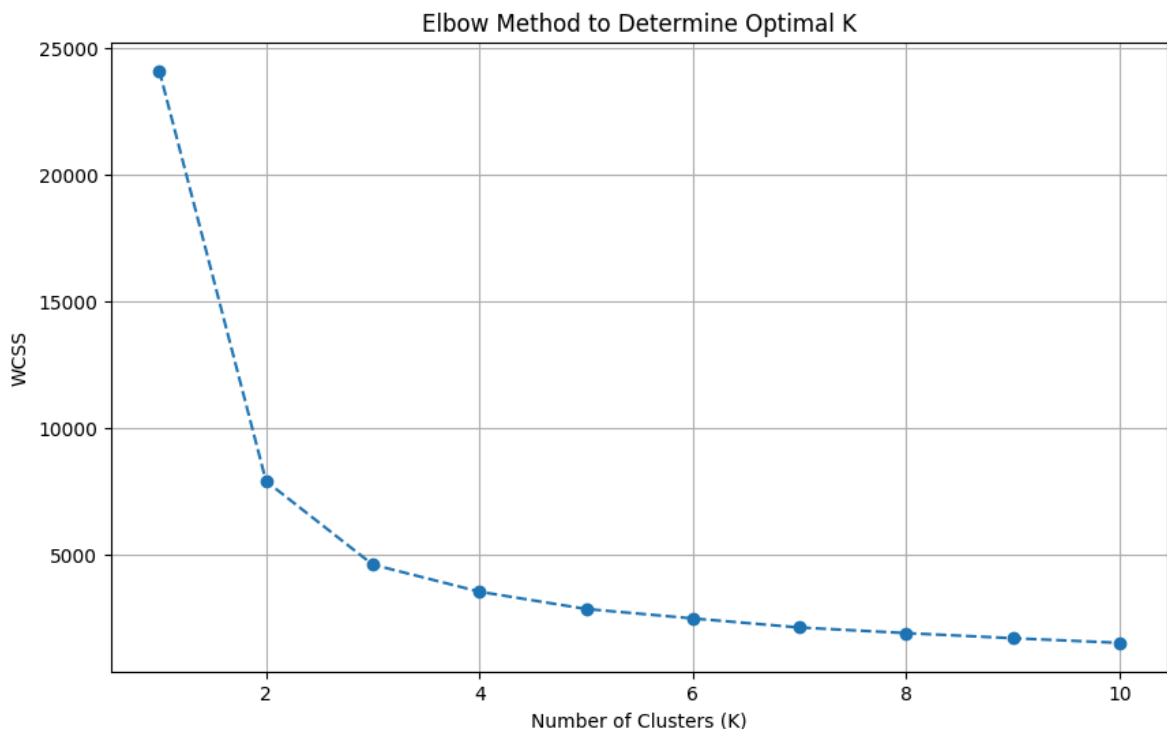
```
from sklearn.cluster import KMeans

# Determine the optimal number of clusters using the Elbow Method
wcss = [] # Within-Cluster Sum of Squares
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
```



```
# Plot the Elbow Method graph
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method to Determine Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```

print("Elbow Method plot generated. Please analyze the plot to choose an opt



Elbow Method plot generated. Please analyze the plot to choose an optimal K.

Based on the Elbow Method plot, let's choose a suitable number of clusters (e.g., K=4 or K=5) for the K-Means algorithm. We will proceed with K=4 for demonstration purposes. You can modify this value if your interpretation of the elbow plot suggests a different optimal number of clusters.

```
# Apply K-Means clustering with the chosen number of clusters
# Let's assume the optimal K is 4 based on the elbow plot analysis
optimal_k = 4 # This can be changed based on the elbow plot interpretation
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_i
clusters = kmeans.fit_predict(X_pca)
```

```
# Add the cluster labels to the PCA DataFrame
pca_df['Cluster'] = clusters

# Add the cluster labels to the original DataFrame for further analysis
data['Cluster'] = clusters

print(f"K-Means clustering applied with {optimal_k} clusters.")
print("Distribution of customers per cluster:")
display(data['Cluster'].value_counts())

# Display the head of the PCA DataFrame with cluster labels
display(pca_df.head())
```

K-Means clustering applied with 4 clusters.

Distribution of customers per cluster:

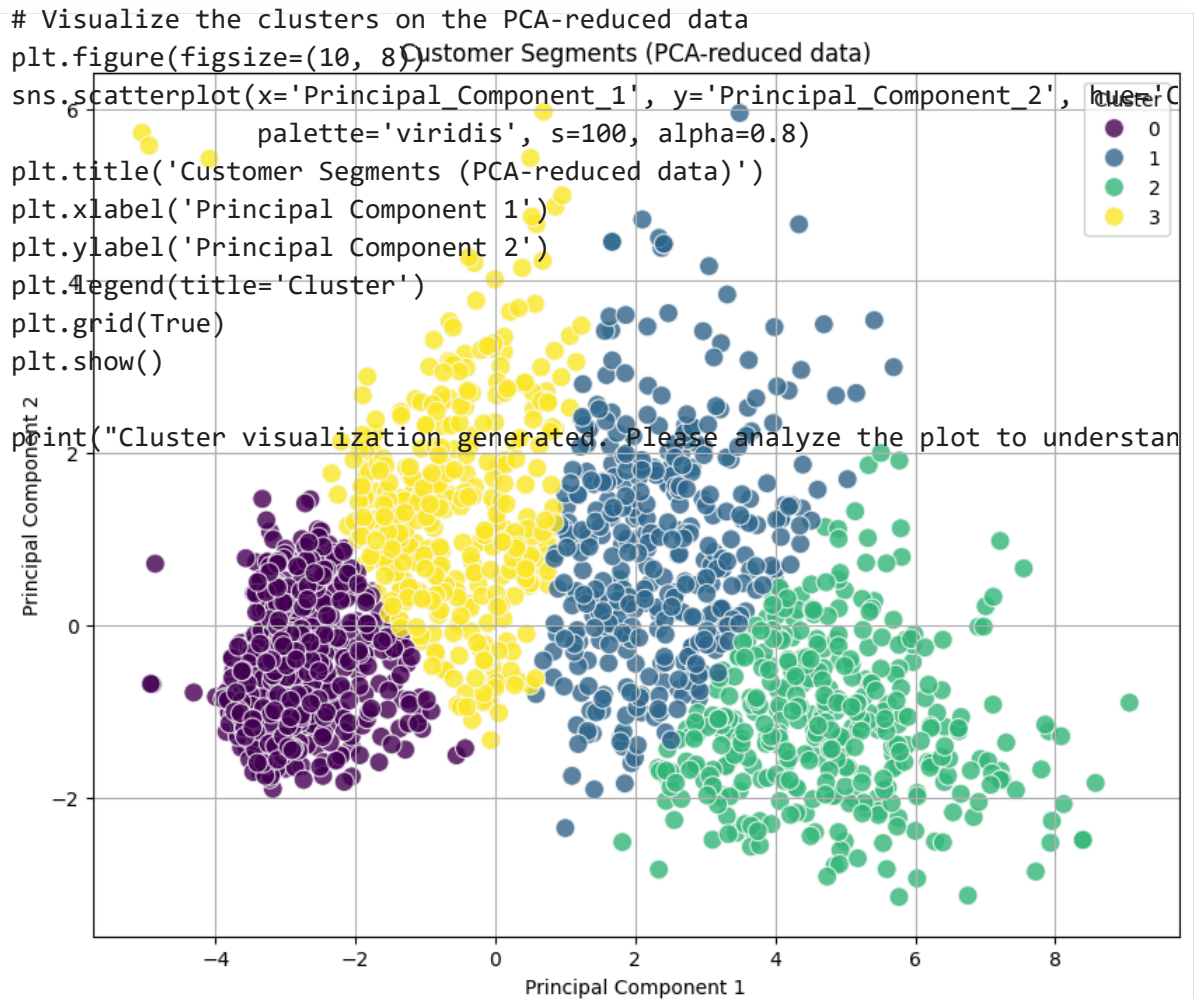
	count
Cluster	
0	863
2	389
1	389
3	380

dtype: int64

	Principal_Component_1	Principal_Component_2	Cluster
0	4.993189	0.411334	2
1	-3.079882	-0.943130	0
2	1.824878	0.232295	1
3	-3.095408	-0.448724	0
4	-0.163483	1.004642	3



✓ Visualization of Customer Segments



Cluster visualization generated. Please analyze the plot to understand the di

✓ Insights and Recommendations: Cluster Profiling